REGISTRO DE MOBILE - SEMANA 23

1. TurboPerformance mobile: análise e otimização de performance em apps

Arquivo: [SIS]ANO2C1B4S23A1AP.docx

1. Estudo inicial — pontos principais

A análise de performance avalia o tempo de carregamento, o consumo de memória, o uso da CPU e da bateria. Ela é importante para garantir que o aplicativo seja rápido, responsivo e ofereça uma boa experiência ao usuário.

Ferramentas como Android Profiler e Xcode Instruments permitem monitorar os recursos do aplicativo, identificar gargalos e analisar logs para encontrar problemas.

2. Aplicação prática — diagnóstico do problema

Problema identificado: um aplicativo de e-commerce apresenta lentidão na exibição dos produtos.

Processo de diagnóstico:

1. Analisar os logs para identificar operações demoradas.
2. Utilizar uma ferramenta de profiling para monitorar o consumo de recursos.
3. Verificar se uma chamada de API não otimizada está causando a lentidão.
4. Identificar possíveis problemas no carregamento das imagens.
5. Aplicar as correções e realizar novos testes para verificar os resultados.

3. Reflexão — estratégias de otimização

As estratégias mais eficazes seriam otimizar as chamadas de API, utilizar lazy loading para carregar os produtos conforme necessário, comprimir e redimensionar imagens e implementar caching para evitar solicitações repetidas. Essas medidas ajudam a reduzir o consumo de recursos e melhorar o tempo de resposta.

4. Perguntas finais

1. Vocês recebem a mensagem de um colega que diz que o carregamento de imagens está comprometendo a performance. O que sugerem?

Sugiro otimizar e comprimir as imagens, utilizar formatos adequados, aplicar lazy loading e armazenar imagens em cache. Também devemos utilizar ferramentas de profiling para verificar o impacto dessas mudanças na performance.

2. Redijam um e-mail ao gestor explicando o diagnóstico e as possíveis soluções para uma chamada de API que compromete o desempenho.

Olá, gestor!

Durante a análise de performance do aplicativo, identificamos que uma chamada de API pode estar causando lentidão no carregamento dos produtos. Para confirmar o problema, recomendamos analisar os logs e utilizar ferramentas de profiling para identificar o tempo de resposta e o consumo de recursos.

Como possíveis soluções, sugerimos otimizar as requisições, reduzir o volume de dados transferidos, implementar caching e evitar chamadas desnecessárias. Após as alterações, realizaremos novos testes para verificar se houve melhora no desempenho e se as funcionalidades continuam funcionando corretamente.

Atenciosamente,
Equipe de Desenvolvimento

3. Como abordar, em uma reunião, relatos de lentidão mesmo após as otimizações?

Devemos coletar mais informações dos usuários, analisar os logs e monitorar o uso de CPU, memória e rede. Também é importante verificar se o problema ocorre em dispositivos específicos ou em determinadas condições de conexão. Com esses dados, podemos identificar a causa e aplicar novas melhorias.

4. Quais pontos destacar no relatório interno de diagnóstico e otimização?

- Problemas identificados e seus impactos.
- Ferramentas utilizadas na análise.
- Causas encontradas nos logs e no profiling.
- Otimizações realizadas.
- Resultados dos testes antes e depois das correções.
- Problemas restantes e recomendações para monitoramento contínuo.

2. Usabilidade em foco: testes práticos para aplicativos móveis

Arquivo: [SIS]ANO2C1B4S23A3AP.docx

1. Estudo inicial — pontos principais

Os testes de usabilidade avaliam a facilidade de uso, a navegação e a satisfação dos usuários. Podem ser qualitativos, como entrevistas e observações, ou quantitativos, como testes A/B, heatmaps e métricas de engajamento.

A análise do feedback permite identificar dificuldades, corrigir problemas e melhorar a experiência do usuário.

2. Planejamento do teste de usabilidade

Cenário: testar um aplicativo de compras pelo celular.

Objetivo: verificar se os usuários conseguem encontrar produtos, consultar informações e finalizar uma compra com facilidade.

Métodos e ferramentas:

- Entrevistas para conhecer a opinião dos usuários.
- Observação durante a navegação.
- Heatmaps para identificar as áreas mais acessadas.
- Testes A/B para comparar diferentes versões da interface.
- Métricas de engajamento para avaliar a interação.

3. Execução e análise dos testes

Os participantes seriam convidados a realizar tarefas, como localizar um produto e acessar suas informações. Durante os testes, seriam registradas as dificuldades de navegação, o tempo para concluir cada tarefa e as opiniões dos participantes.

Exemplo de resultado simulado: os usuários apresentam dificuldades para encontrar a busca e localizar o botão de finalizar compra.

Melhorias propostas: destacar a barra de busca, simplificar os menus, tornar os botões mais visíveis e reduzir etapas desnecessárias. As alterações seriam testadas novamente para verificar se facilitaram a navegação.

4. Perguntas finais

1. Após os testes, os usuários estão se confundindo com a navegação. Como podemos resolver isso?

Devemos analisar as dificuldades relatadas, observar como os usuários navegam e utilizar heatmaps para identificar pontos problemáticos. Depois, podemos simplificar os menus, melhorar a organização das telas e destacar os botões principais. Por fim, devemos realizar novos testes para verificar se as mudanças melhoraram a usabilidade.

2. Redijam uma resposta ao gerente de produto propondo melhorias com base no feedback dos usuários.

Olá, gerente de produto!

Com base no feedback coletado durante os testes de usabilidade, recomendamos priorizar os problemas que dificultam a navegação e a conclusão das tarefas.

As principais estratégias são simplificar os menus, melhorar a visibilidade dos botões, organizar melhor as informações e reduzir etapas desnecessárias. Também sugerimos realizar novos testes com usuários, utilizando entrevistas, observação e heatmaps para avaliar os resultados.

Após a implementação, acompanharemos as métricas de engajamento e as opiniões dos usuários para verificar a eficácia das melhorias.

Atenciosamente,
Equipe de Desenvolvimento

3. Como utilizar o feedback dos testes para priorizar melhorias?

Devemos priorizar os problemas que afetam mais usuários, dificultam tarefas importantes ou provocam abandono do aplicativo. Também precisamos considerar a frequência das reclamações e o impacto de cada problema na experiência do usuário.

4. Quais pontos destacar no relatório dos testes e como recomendar o acompanhamento das melhorias?

O relatório deve apresentar os objetivos, os métodos utilizados, as tarefas avaliadas, as dificuldades encontradas, os feedbacks recebidos e as melhorias propostas. Recomenda-se estabelecer prioridades, implementar as alterações e realizar novos testes, acompanhando métricas de engajamento e satisfação dos usuários.

3. Garantindo perfeição: automatizando publicação nas stores

Arquivo: [SIS]ANO2C1B4S23A4AP.docx

1. Estudo inicial — pontos principais

As ferramentas de build, como Gradle, Xcode Build System e Fastlane, permitem compilar, empacotar e preparar aplicativos para publicação.

A automação de release utiliza scripts e pipelines de CI/CD para executar etapas como compilação, testes, geração dos arquivos de publicação e validação. Isso reduz erros manuais, acelera as entregas e torna o processo mais consistente.

2. Planejamento do pipeline de CI/CD

O fluxo de automação será organizado nas seguintes etapas:

1. Envio do código: a equipe envia as alterações ao repositório.
2. Build: o sistema compila e empacota o aplicativo utilizando as ferramentas adequadas.
3. Testes automatizados: são executados testes para identificar erros.
4. Validação: a equipe verifica os resultados e os requisitos de qualidade.
5. Geração dos artefatos: são preparados os arquivos necessários para a publicação.
6. Release: a versão aprovada é enviada para a etapa de publicação na loja correspondente.
7. Monitoramento: são acompanhados os resultados da publicação e possíveis problemas.

3. Simulação e análise do pipeline

Cenário: durante a automação da publicação, os testes falham e impedem a geração de uma versão válida do aplicativo.

O pipeline deve interromper o processo de publicação, registrar os erros e informar a equipe. Os desenvolvedores analisam os logs, corrigem os problemas e executam novamente os testes. Somente após a aprovação das verificações o build deve seguir para a etapa de publicação.

Esse procedimento evita que versões com erros sejam liberadas aos usuários.

4. Perguntas finais

1. O pipeline apresenta problemas porque os testes não estão passando. Qual seria a abordagem para resolver isso?

Primeiro, analisaria os logs para identificar quais testes falharam e suas possíveis causas. Em seguida, corrigiria os erros no código ou na configuração do ambiente, executaria os testes novamente e verificaria se o build foi gerado corretamente. A publicação só continuaria após a aprovação das verificações.

2. Redijam um e-mail explicando como a automação do build e release reduz o tempo de publicação e minimiza erros manuais.

Olá, gerente de TI!

A automação dos processos de build e release permite executar tarefas como compilação, testes e preparação dos arquivos de publicação de maneira padronizada.

Com ferramentas como Gradle, Xcode Build System e Fastlane, integradas a pipelines de CI/CD, é possível reduzir tarefas manuais, acelerar as entregas, identificar erros antecipadamente e garantir maior consistência entre as versões.

Além disso, recomendamos manter testes automatizados, etapas de validação e monitoramento contínuo para garantir que somente versões aprovadas avancem para a publicação.

Atenciosamente,
Equipe de Desenvolvimento

3. Como garantir a segurança e a integridade do aplicativo durante o release automatizado?

Devemos proteger as credenciais e as chaves de assinatura, restringir os acessos ao pipeline, utilizar dependências confiáveis e executar testes de segurança. Também é importante validar os artefatos gerados, manter registros das alterações e permitir a publicação somente após as verificações necessárias.

4. Quais pontos críticos destacar no relatório e como recomendar o monitoramento contínuo?

O relatório deve destacar as etapas do pipeline, as ferramentas utilizadas, os resultados dos testes, os erros encontrados, os critérios de aprovação e os artefatos gerados. Recomenda-se monitorar continuamente as falhas de build, o tempo de execução, os resultados dos testes e os problemas de publicação, registrando ocorrências e aplicando correções sempre que necessário.